Relazione del progetto di Programmazione a Oggetti

(aa. ss. 2024-25)

Stellin Elia Ernesto (mat. 2101084)

1 Introduzione

Il software sviluppato, "StarLib" è una libreria virtuale che permette di creare, modificare, visualizzare, cercare e cancellare degli articoli dentro delle librerie personali, quali libri, film e articoli di riviste.

StarLib offre una gestione immediata e semplice delle librerie virtuali personali, codificate tramite file JSON, ispirandosi a software di gestione di libri virtuali, quali Calibre. Inoltre, il software offre anche ulteriori funzionalità quali la ricerca avanzata di elementi all'interno della libreria, una modifica granulare dei metadati di ciascun articolo, il salvataggio di alcuni articoli come "preferiti", la gestione di librerie multiple e altro ancora.

Il punto di forza del software è la sua interfaccia grafica moderna e intuitiva che fornisce all'utente una User Experience accessibile e piacevole.

2 Descrizione del modello

Il modello logico del software è diviso in più parti:

- Il modello per descrivere ciascun tipo di articolo,
- Le classi utilizzate per descrivere una libreria e serializzarla in JSON
- Classi aggiuntive che permettono il salvataggio delle impostazioni tramite JSON

Si noti che tutte queste classi appartengono al namespace `Model`

2.1 Modello per i tipi di articolo

Il modello logico principale su cui si basa l'applicazione gestisce i vari tipi di articoli che possono essere registrati dentro una libreria.

Il modello parte da un supertipo astratto `AbstractArticle`, da cui tutti gli articoli derivano, che racchiude le informazioni comuni a tutti i tipi di articolo, come il titolo, la descrizione, l'appartenenza dell'articolo agli elementi preferiti e un'eventuale data di rilascio. Per tutti questi attributi vengono definiti i relativi getter e setter; molti di questi attributi sono opzionali, in particolare la data di rilascio dell'articolo (`AbstractArticle::release_date`) è descritta col tipo `std::optional<QDateTime>`.

Si noti che ciascun `AbstractArticle` definisce anche un attributo `_uuid`, usando il tipo `QUuid` offerto da Qt, per poter ricercare in tempo costante gli articoli all'interno di un file di libreria (vd. Sez. 4). Inoltre, viene definito il metodo `const QString& AbstractArticle::getArticleDisplayType()` per mostrare il tipo di un articolo all'interno di label testuali, ma non come meccanismo di RTTI.

La classe `Movie` è la prima classe concreta che deriva da `AbstractArticle`, a questa sono associati vari attributi aggiuntivi, tra cui `length` di tipo `const Helper::MediaRuntime&`, un tipo definito per rappresentare la lunghezza di qualunque medium audio-visivo.

`MediaRuntime` è stata creata per poter essere riutilizzata, in futuro, per composizione in altri tipi di articolo che necessitano di descrivere una durata di tempo (per es., per descrivere delle canzoni, dei podcast o degli episodi di una serie televisiva). È stato scelto di creare questa classe, invece che usarne altre nella STL o QTL (ad es., `std::chrono::duration`), per avere a disposizione una classe semplice e riutilizzabile.[1]

[1] L'ispirazione è stata `deltatime` di Python, che offre un modo estremamente semplice di rappresentare le durate temporali, mentre, per es. la classe `std::chrono::duration` della STL sembrava troppo complicata da utilizzare.

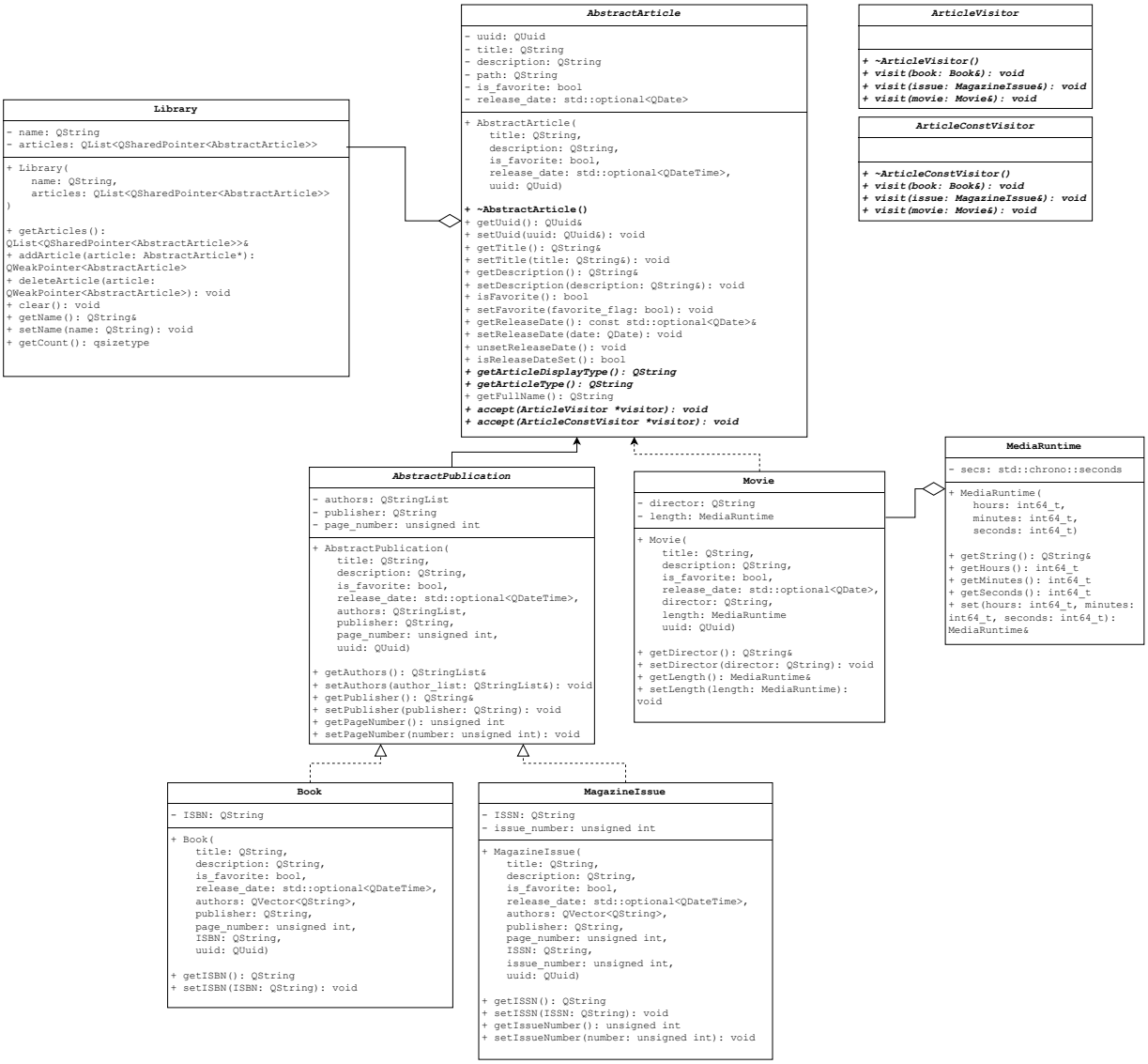

Figura 1: Schema UML del modello logico del software

Successivamente, viene descritta un'ulteriore classe astratta `AbstractPublication` per descrivere una pubblicazione qualunque, qui si descrivono informazioni quali gli autori, la casa editrice e il numero di pagine, insieme ai relativi getter e setter.

Da `AbstractPublication` derivano le classi concrete `Book` e `MagazineIssue`, che rappresentano rispettivamente un libro e un'uscita di una rivista. La differenza tra queste classi è il fatto che possiedano attributi diversi per rappresentare il loro codice univoco di prodotto (`ISBN` per `Book` e `ISSN` per `MagazineIssue`) e il fatto che `MagazineIssue` possa descrivere il proprio numero di serie come attributo (`issue_number`). Si noti che la classe `AbstractPublication` è stata creata con lo scopo di essere estesa da altre classi concrete future, non ancora descritte dal software (quali, per es. giornali online, articoli scientifici, etc).

2.2 Modello per la libreria e la serializzazione

Il software utilizza la classe `Library` per rappresentare una libreria virtuale.

Le caratteristiche principali di una libreria sono il suo nome, rappresentato da una `QString` e la lista degli articoli al suo interno, rappresentati da un attributo di tipo `QList<QSharedPointer<AbstractArticle>>`, sfruttando il design pattern *Composite*. L'utilizzo dei puntatori condivisi per rappresentare gli articoli è

ubiquitario nel software per prevenire il più possibile memory leak e altri bug relativi alla memoria.[2] Per questi attributi vengono, inoltre, definiti i corretti getter e setter. Inoltre, sono stati implementati metodi per ottenere la lista di tutti gli articoli all'interno di una libreria, aggiungervi un nuovo articolo o rimuoverne uno già presente.

Per quanto riguarda la serializzazione delle librerie, si utilizzano le classi `DataMapper::JsonFile` e le funzioni helper definite nel namespace `Model::Converter`.

La classe `JsonFile` rappresenta il file JSON utilizzato per salvare su disco i dati di una libreria, offrendo le funzionalità di aggiunta, rimozione e modifica di file a esso associati. Il salvataggio di tutte le modifiche apportate al file JSON può avvenire automaticamente, se si imposta il salvataggio automatico tramite `setAutosave(true)` o manualmente tramite il metodo `save()`.

Le funzioni nei namespace `Model::Converter::JsonReader` e `Model::Converter::Json` permettono, rispettivamente, di leggere da un file di libreria JSON e di convertirlo nel corrispettivo oggetto `Library`, tramite l'utilizzo della classe `QJsonObject` offerta da Qt e di visitor creati appositamente che verranno discussi successivamente.

Analogamente, si utilizzano gli stessi strumenti nella classe `Model::Config::Settings` per salvare su disco le impostazioni a discrezione dell'utente (vd. sez 4).

3 Polimorfismo

L'utilizzo principale del polimorfismo all'interno del software si ha nell'implementazione del design pattern *Visitor* nella gerarchia degli articoli.

Vengono utilizzati diversi Visitor per compiere diverse funzionalità:

- `ArticlePreviewWidget`: Creare widget di preview per ogni tipo di articolo
- `ArticleInfoWidget`: Creare le pagine di visualizzazione dei dati per ogni tipo di articolo
- `ArticleEditInfoWidget`: Creare i form per la modifica di ciascun tipo di articolo
- `EditArticleVisitor`: Modificare i dati di ogni tipo diverso di articolo, a seconda dei dati inseriti dall'utente nel form di modifica
- `QueryMatchVisitor`: Verificare se un articolo appartiene a una ricerca compiuta dall'utente
- `JsonVisitor`: Converte un articolo nella sua rappresentazione JSON

In generale, ciascuna classe Visitor eredita da una tra le classi astratte `ArticleVisitor` o `ArticleConstVisitor`.

Se il visitor ha la possibilità di modificare un articolo allora eredita da `ArticleVisitor`, altrimenti se deve solamente leggerne i contenuti eredita da `ArticleConstVisitor`

3.1 ArticlePreviewWidget

`ArticlePreviewWidget` è un Visitor che eredita da `ArticleConstVisitor` e restituisce un widget di anteprima da mostrare nella griglia principale del programma.

Il widget mostrato in anteprima è diverso graficamente a seconda del tipo di articolo che rappresenta:

- Per gli articoli di tipo `Book`, vengono mostrati il titolo e l'autore del libro,
- Per i `MagazineIssue`, vengono mostrati il titolo e il numero d'uscita dell'uscita della rivista,
- Per i `Movie`, vengono mostrati il titolo e la durata in ore, minuti e secondi del film.

`ArticlePreviewWidget` sfrutta a sua volta il design pattern *Composite*, in quanto contiene un attributo `widget` di tipo `ArticlePreviewBase` per rappresentare il widget che verrà utilizzato come anteprima nella griglia di ricerca. Quest'ultima, infatti, è una sottoclasse di `QWidget` che contiene tutti gli elementi comuni ai vari widget di anteprima costruibili, ovvero:

- Il `QFrame` quadrato stilizzato che contiene tutti i dati di anteprima,
- Il bottone dei preferiti di tipo `FavoriteButton`, che è responsivo, ovvero si aggiorna ogni qualvolta che un articolo viene aggiunto tra i preferiti. Tale azione può essere compiuta, infatti, anche tramite una shortcut apposita (Ctrl + P) oppure tramite il bottone nella pagina informativa dell'articolo, presente sul lato destro dello schermo,

[2] Difatti, poiché la libreria è il "proprietario" principale degli articoli, quando un oggetto `AbstractArticle` viene distrutto, perché rimosso dalla libreria o perché lo stesso oggetto `Library` viene distrutto, ogni puntatore condiviso all'articolo viene invalidato, evitando situazioni in cui si presentano "dangling pointer".

- Lo stile personalizzato e la *business logic* del componente.

In generale, questo widget permette di essere riutilizzato in modo molto ampio, applicando il principo DRY (Don't Repeat Yourself).

3.2 ArticleInfoWidget

`ArticleInfoWidget` eredita anch'esso da `ArticleConstVisitor` e mostra la pagina di informazioni di un articolo, sulla metà destra della finestra dell'applicazione, tramite dei layout di tipo form.

Questo visitor crea di form appositi per tutti i tipi di articoli salvabili, i quali possiedono attributi diversi. Per cui, grazie a questo visitor, un articolo di tipo `Book` non potrà mostrare informazioni a esso irrilevanti o insensate, quali il "regista" o la "durata".

3.3 ArticleEditInfoWidget

`ArticleEditInfoWidget` eredita da `ArticleConstVisitor` e crea i form di modifica per ciascun tipo di articolo.

Anche in questo caso, poiché ogni tipologia di articolo ha attributi diversi, sarà possibile solamente modificare gli attributi rilevanti a ciascun articolo.

Si noti che la creazione del form di modifica avviene tramite delle funzioni helper nel namespace `View::Form` che permettono di creare il form di modifica per ciascun tipo di articolo (`createBookForm`, `createMovieForm` e `createIssueForm`), ma non implementando il design pattern *Visitor*. In ciascuna di queste funzioni, a ogni elemento di input del form si associa un nome nel framework Qt usando il metodo `QWidget::setObjectName()`, in modo tale da rendere più facile la loro lettura successivamente.

3.4 EditArticleVisitor

`EditArticleVisitor`, invece eredita da `ArticleVisitor` poiché modifica l'articolo che visita a seconda delle informazioni lette da un form di modifica, come quello descritto in precedenza nella sezione 3.3.

Più nello specifico, il visitor è costruito prendendo in input una variabile di tipo `QWidget*` che punta al form dal quale leggere i nuovi dati dell'articolo. Quindi, l'articolo viene modificato leggendo i dati rilevanti, presenti negli elementi di input del form ricevuto.

Anche in questo caso, l'utilizzo del design pattern *Visitor* permette a `EditArticleVisitor` di leggere gli attributi rilevanti del form per ogni tipo di articolo.

3.5 QueryMatchVisitor

`QueryMatchVisitor` eredita da `ArticleConstVisitor` e determina se un articolo corrisponda a una query di ricerca all'interno della libreria, rappresentata dalla classe `Model::Search::Query`.

Nello specifico, una query di ricerca è composta da:

- Il testo che l'utente ha inserito nella barra di ricerca,
- Una flag booleana impostata a `true` se l'utente sta cercando articoli di tipo `Book`,
- Una flag booleana analoga per la classe `Movie`,
- Una flag booleana analoga per la classe `MagazineIssue`,
- Una flag booleana che è impostata a `true` se l'utente sta cercando solo tra i suoi articoli preferiti e a `false` se la ricerca può comprendere anche gli articoli non preferiti.

Il design pattern *Visitor* permette a `QueryMatchVisitor` di determinare se un articolo sia di una tipologia desiderata o meno dall'utente, senza usare `dynamic_cast` o altri meccanismi di RTTI.

3.6 JsonVisitor

Infine, `JsonVisitor` eredita da `ArticleConstVisitor` e permette di convertire un articolo del modello logico nella sua rappresentazione JSON, usata successivamente durante la fase di serializzazione della libreria.

Il design pattern *Visitor* permette di convertire ciascun articolo nel modo corretto, assicurando che la rappresentazione JSON di ciascun articolo possieda gli attributi strettamente necessari e rilevanti.

4 Persistenza dei dati

Per quanto riguarda la persistenza dei dati, una libreria è salvata su disco in formato JSON: più nello specifico, a un file JSON corrisponde una singola libreria virtuale.

Il contenuto del file è un oggetto JSON, ovvero una mappa chiave-valore, con la seguente struttura:

- Alla chiave `"name"` si associa una stringa che contiene il nome della libreria, modificabile dall'utente,
- Alla chiave `"articles"`, invece, si associa un sotto-oggetto che a sua volta associa a un UUID un articolo, rappresentato da un oggetto con la seguente struttura:
  - Alla chiave `"type"` si associa il tipo dell'articolo in questione (può essere `"book"` per i libri, `"issue"` per le uscite di riviste o `"movie"` per i film)
  - Le restanti chiavi rappresentano gli attributi dell'articolo, e dipendono dal tipo di articolo in sé, quali il titolo, la descrizione e tutti gli attributi specifici di ciascun tipo di articolo.[3]

Un esempio della struttura del formato JSON è dato dal file `Libreria.json`, fornito insieme al codice sorgente, che contiene tutti i tipi di articolo, in modo tale da illustrare brevemente tutte le possibili sotto-strutture usate dal formato.

Inoltre, si utilizza il formato JSON per salvare le impostazioni definite dall'utente, quali l'ultima libreria aperta dall'utente e un flag che determina se usare il salvataggio automatico.

Queste informazioni vengono salvate nel file `~/Library/Preferences/StarLib/config.json` su Linux e `%localappdata%/StarLib/config.json` su Windows, il quale viene modificato automaticamente ogni volta che queste impostazioni vengono cambiate.

5 Funzionalità implementate

Le funzionalità implementate dal software sono di seguito suddivise tra funzionalità pratiche, ovvero relative alle azioni che possono essere compiute dall'utente e funzionalità puramente estetiche, implementate per migliorare la UX.

5.1 Funzionalità pratiche

1. Gestione di tre tipi di articoli:
   - Aggiunta
   - Modifica
   - Rimozione dalla libreria
   - Possibilità di aggiungere articoli tra i "preferiti"
2. Gestione delle librerie:
   - Creazione di nuove librerie
   - Possibilità di aprire file di libreria, trascinando un file JSON valido oppure aprendolo usando il menù delle azioni
   - Modifica dei dati della libreria
3. Ricerca di articoli all'interno di una libreria
   - Ricerca testuale parziale e case-insensitive
   - Ricerca avanzata per tipo
   - Ricerca tra gli elementi preferiti
4. Conversione e salvataggio in formato JSON

5.2 Funzionalità estetiche

1. Stile grafico personalizzato a tema scuro
2. Shortcut per ogni azione principale
3. Barra dei menù nella parte alta dello schermo:
   - Nella barra è presente ognuna delle possibili azioni che l'utente può compiere
   - Molte azioni sono accompagnate dalla relativa shortcut

[3] Infatti, in un articolo di tipo `"book"` probabilmente non comparirà il campo `"directors"` a meno che il file JSON non sia stato modificato dall'utente. Anche in questa eventualità, però, il valore irrilevante verrebbe ignorato dal software e non causerebbe alcun problema.

- Molte azioni sono accompagnate da un'icona rappresentativa

4. Gestione del salvataggio:
    1. Salvataggio automatico (attivabile via menù)
    2. Controllo delle modifiche non salvate prima di uscire o prima di aprire una nuova libreria, se il salvataggio automatico è disattivato
    3. Apertura dell'ultima libreria aperta allo startup dell'applicazione
5. Icona dell'applicazione personalizzata
6. Schermata di benvenuto quando nessuna libreria è aperta
7. Status bar nella parte bassa della finestra, che descrive l'ultima azione compiuta
8. Ogni operazione è condotta in una singola schermata
9. Divisione dello schermo tramite splitter ridimensionabile dall'utente
    - Parte sinistra dello schermo: griglia di visualizzazione di articoli e barra di ricerca avanzata
    - Parte destra dello schermo: pagina di visualizzazione dei dettagli di un articolo, form di modifica degli stessi o form di aggiunta di un nuovo articolo

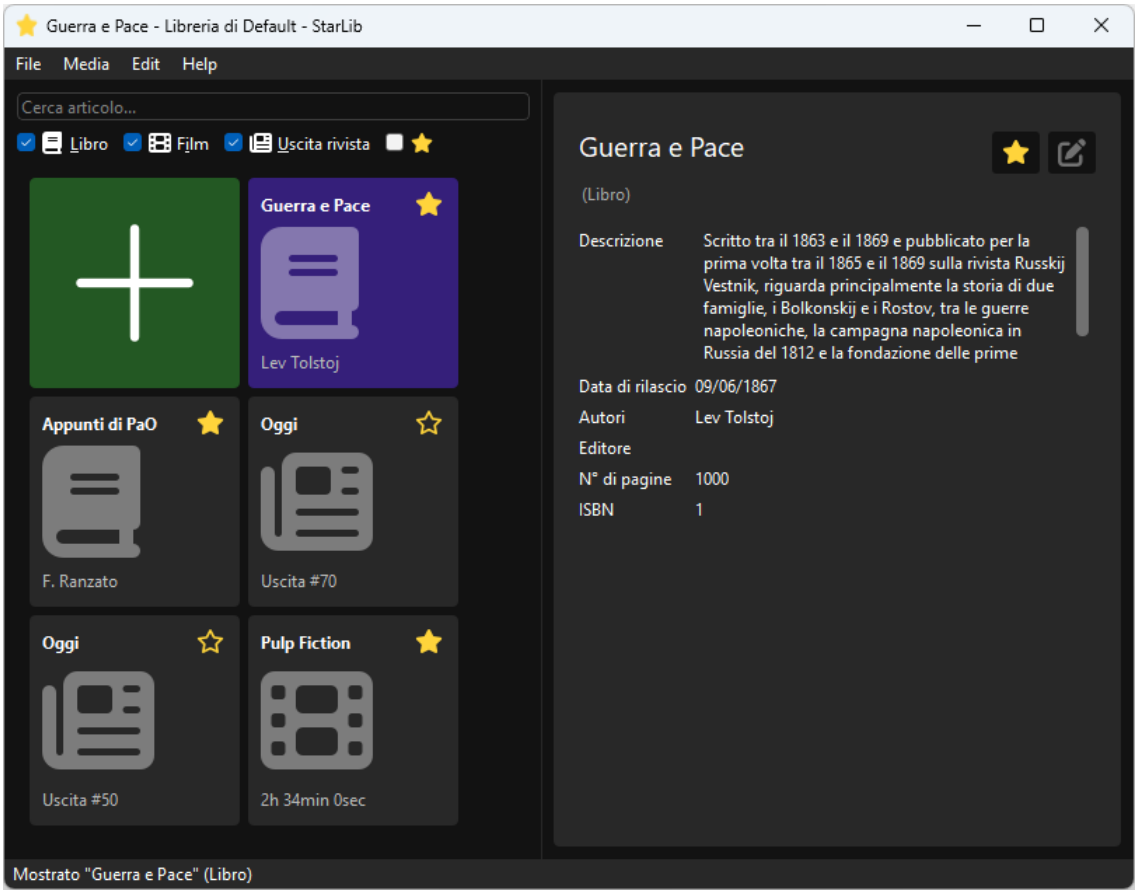

Figura 2: Schermata principale del Software: si noti la divisione dello schermo nelle due parti.

10. Visualizzazione a griglia degli articoli (parte sinistra)
    1. Il primo elemento della griglia è sempre un bottone verde che mostra il form di aggiunta articoli quando cliccato
    2. Ogni elemento dell'articolo è rappresentato con un riquadro nella griglia, diverso per ogni tipo di articolo, come già spiegato nella sezione 3.2
    3. Cliccando il riquadro, viene mostrato l'articolo e il riquadro cambia colore
    4. Nel riquadro è presente il bottone "preferiti"

5. Vengono gestiti i context menu per gli elementi della griglia, cliccando tasto destro su uno di essi

11. Interfaccia grafica responsiva
    1. Se un articolo viene inserito tra i “preferiti”, la modifica si riscontra direttamente nei bottoni dei “preferiti” della griglia
    2. Barra di ricerca responsiva: quando viene inserita una query, la griglia di visualizzazione si aggiorna automaticamente mostrando i risultati rilevanti, secondo quanto descritto nella sezione 3.5.
12. Gestione della paginazione sul lato destro dello schermo

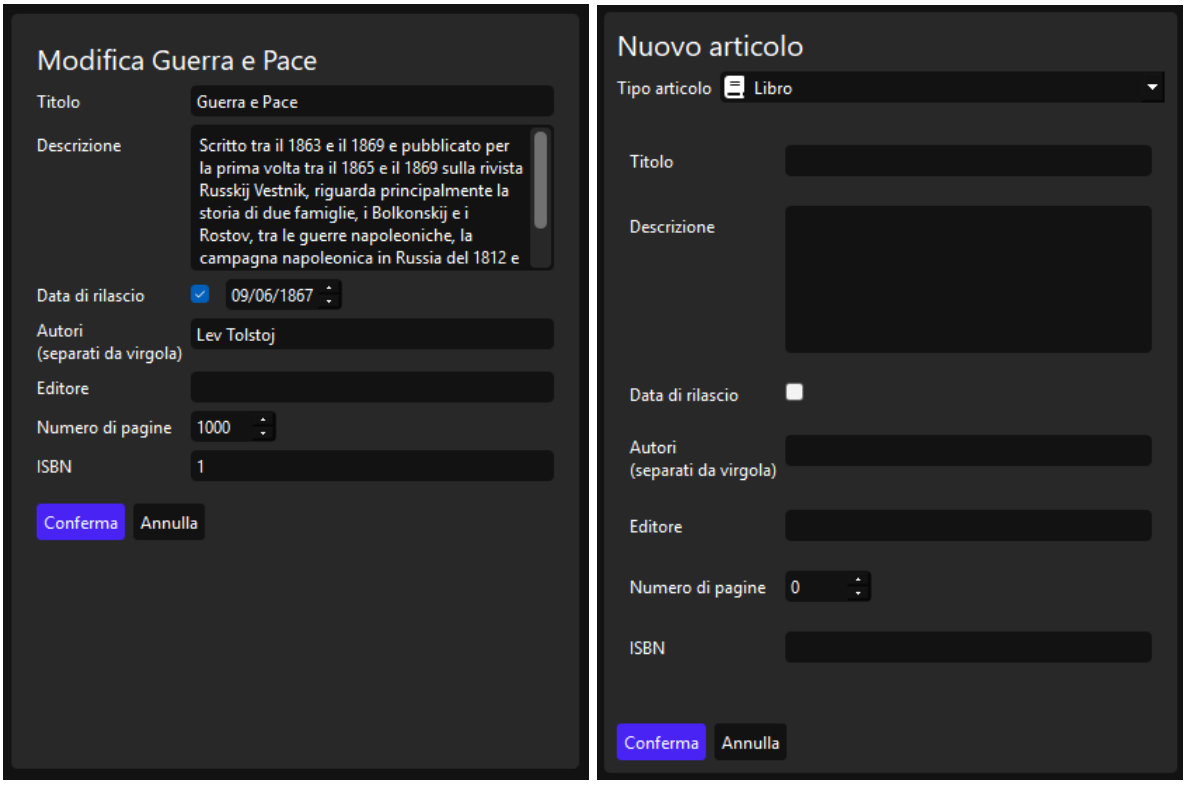

Figura 3: A sinistra il form di modifica di un articolo, a destra il form di creazione di un nuovo articolo.

13. Gestione del ridimensionamento della finestra
14. Menù “About”, accessibile tramite menù

6 Rendicontazione ore

Attività	Ore previste	Ore effettive
Studio e progettazione	8	6
Sviluppo del modello logico	7	7
Studio del framework Qt	5	8
Sviluppo della GUI	20	45
Test e debug	5	4
Stesura della relazione	5	5
Totale	**50**	**75**

La motivazione per cui il conteggio delle ore effettive abbia sforato è spiegata meglio nella prossima sezione. In breve, però, ciò è dovuto al fatto che imparare il framework Qt abbia richiesto più tempo del previsto e lo sviluppo dell'interfaccia sia diventato prevalente, in quanto è fondamentale per ogni applicazione desktop.

7 Note aggiuntive

Come si può notare anche dalla rendicontazione delle ore, l'interfaccia grafica dell'applicazione è più avanzata rispetto a quanto richiesto dal progetto, sia da un punto di vista puramente visivo, che da un punto di vista implementativo.

La decisione di aggiungere questa grande quantità di funzionalità prettamente estetiche è stata presa, poiché l'interfaccia grafica è il modo con cui lo sviluppatore e l'applicazione comunicano direttamente con l'utente finale, il quale molto spesso non è un utente tecnico. Per tale motivo, è fondamentale che la UI di qualunque software sia il più semplice da utilizzare e piacevole da vedere, in modo tale da essere accessibile a più utenti possibili.

Dal punto di vista implementativo, invece, l'implementazione della UI sarebbe stata più veloce utilizzando il design pattern *Observer* per collegare i vari elementi responsivi della UI: invece il progetto ha usato una grande quantità di segnali connessi tra loro dalla funzione `connect()` offerta da Qt.

Inizialmente il progetto aveva una dimensione tale da rendere l'implementazione del pattern *Observer* uno spreco di tempo, ma col crescere dello *scope* del software, sarebbe invece stato utile per connettere più facilmente gli elementi della UI. Alla fine, però, migrare dal sistema di slot e segnali al sistema *Observer* avrebbe comunque richiesto troppo tempo, per cui è stato deciso di usare il sistema vecchio fino alla fine.

L'utilizzo di quest'ultimo, però, richiedeva la scrittura di molto codice "boilerplate", contribuendo ad aumentare la quantità di tempo spesa a scrivere il codice della GUI

Oltre a tutto ciò, imparare a utilizzare il framework Qt ha richiesto ulteriore tempo, essendo un framework complesso e molto esteso.